



Universidad Autónoma del Estado de Hidalgo
Licenciatura en Ciencias Computacionales
Automatas y Compiladores
Docente: Dr.Eduardo Cornejo Velazquez

REPORTE DE PRÁCTICA NO.7:

Análisis sintáctico de elementos de un lenguaje de programación

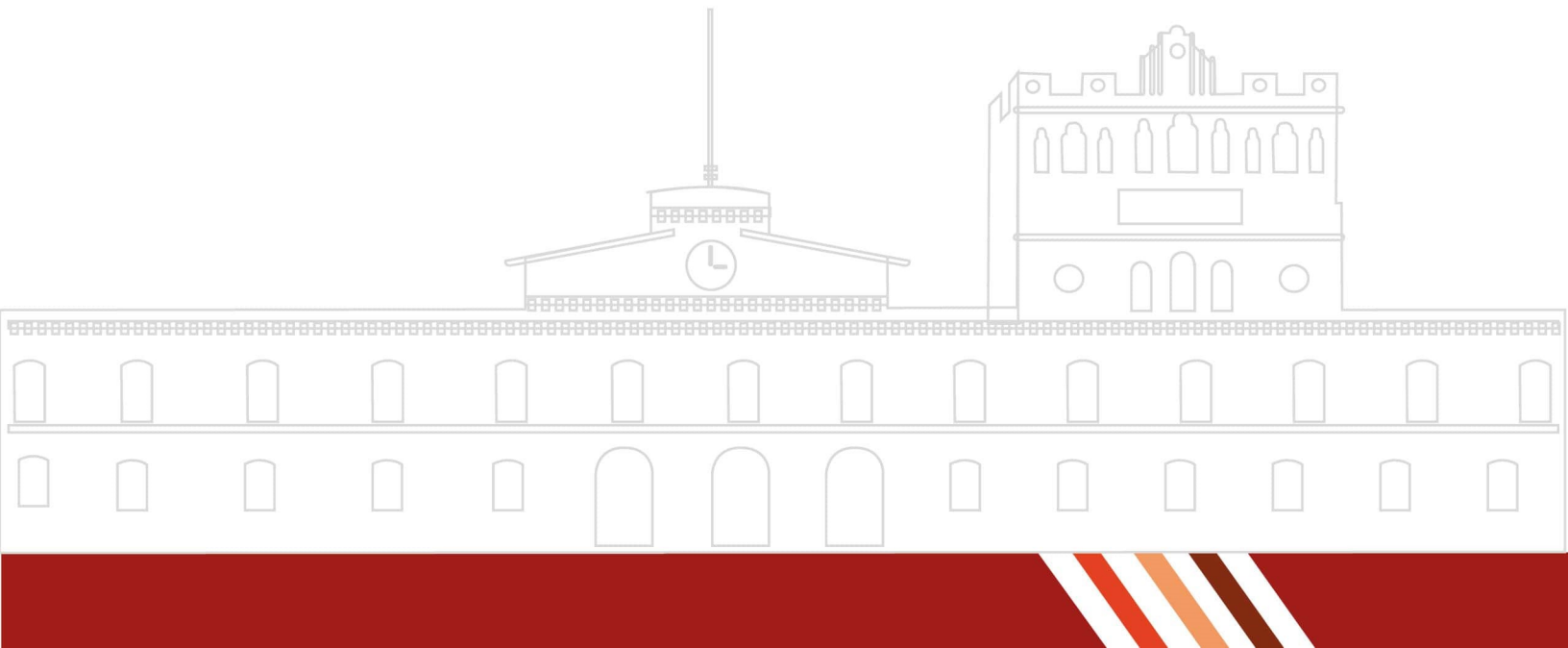
ALUMNO:

Naomi Romero López

Ángel Ortega Gómez

Adrian Hernández Sánchez

Luis Jose Moreno Ortega



1. Introducción

El análisis de lenguajes de programación constituye una parte fundamental en el desarrollo de compiladores, ya que permite interpretar y validar la estructura de los programas escritos por el usuario. Dentro de este proceso, el análisis léxico y sintáctico desempeñan un papel clave al encargarse de reconocer los elementos básicos del lenguaje y verificar que su organización cumpla con las reglas definidas por una gramática formal.

En esta práctica se aborda la construcción de un analizador léxico y un analizador sintáctico utilizando las herramientas Lex y Yacc, las cuales facilitan la generación automática de estos componentes a partir de especificaciones formales. El analizador léxico se encarga de identificar los tokens presentes en una estructura condicional, mientras que el analizador sintáctico valida que dichos elementos se encuentren organizados correctamente conforme a una gramática de contexto libre.

El desarrollo se centra en el reconocimiento de la estructura condicional `if - then`, una de las construcciones más representativas en los lenguajes de programación de alto nivel. A partir de la definición de autómatas, expresiones regulares y reglas de producción, se implementa un sistema capaz de detectar tanto estructuras válidas como errores sintácticos, permitiendo comprender de manera práctica el funcionamiento interno de estas herramientas.

Finalmente, se realizan pruebas con distintos ejemplos de código para analizar el comportamiento del sistema desarrollado, verificando su capacidad para identificar correctamente las estructuras válidas y señalar errores en casos incorrectos.

2. Marco teórico

Análisis léxico

El análisis léxico constituye la primera fase en el proceso de compilación, cuya función principal es transformar una secuencia de caracteres en un conjunto de unidades significativas denominadas tokens. Estos tokens representan elementos básicos del lenguaje como identificadores, palabras reservadas, operadores y constantes. Esta etapa simplifica el trabajo del analizador sintáctico al estructurar la entrada de manera organizada [1].

El analizador léxico se basa en el uso de expresiones regulares y autómatas finitos para reconocer patrones dentro del código fuente. Herramientas como Lex permiten automatizar este proceso mediante la definición de reglas que asocian patrones con acciones específicas, facilitando la generación de analizadores eficientes [2].

Análisis sintáctico

El análisis sintáctico es la fase encargada de verificar que la secuencia de tokens generada por el análisis léxico cumpla con las reglas gramaticales del lenguaje. Para ello, se utilizan gramáticas formales, particularmente gramáticas libres de contexto, que permiten describir la estructura jerárquica de las instrucciones [1].

Los analizadores sintácticos pueden clasificarse en descendentes y ascendentes. Herramientas como Yacc implementan analizadores ascendentes del tipo LR, los cuales son capaces de manejar una amplia variedad de gramáticas y detectar errores sintácticos de manera eficiente durante el proceso de análisis [2].

Gramáticas formales

Una gramática formal es un modelo matemático que permite describir la estructura de un lenguaje mediante reglas bien definidas. Se representa como una tupla $G = (\Sigma_T, \Sigma_N, S, P)$, donde Σ_T corresponde a los símbolos terminales, Σ_N a los símbolos no terminales, S es el símbolo inicial y P es el conjunto de producciones.

Estas reglas permiten generar cadenas válidas a partir del símbolo inicial, constituyendo la base para el análisis sintáctico en los lenguajes de programación [3].

Gramáticas libres de contexto

Las gramáticas libres de contexto son un tipo particular de gramáticas formales en las cuales cada producción tiene un único símbolo no terminal en el lado izquierdo. Este tipo de gramáticas es especialmente útil para describir la sintaxis de los lenguajes de programación, ya que permite representar estructuras anidadas como expresiones aritméticas o estructuras de control [4].

Además, las gramáticas libres de contexto facilitan la construcción de árboles de derivación, los cuales representan la forma en que una cadena es generada a partir del símbolo inicial, permitiendo visualizar la estructura sintáctica del programa [1].

Derivaciones

Las derivaciones son el proceso mediante el cual se generan cadenas a partir del símbolo inicial aplicando reglas de producción. Existen dos tipos principales: derivación por la izquierda y derivación por la derecha. En la derivación por la izquierda se sustituye siempre el símbolo no terminal más a la izquierda, mientras que en la derivación por la derecha se sustituye el símbolo más a la derecha [3].

Estas derivaciones son fundamentales para el análisis sintáctico, ya que permiten determinar si una cadena pertenece o no al lenguaje definido por una gramática [4].

Herramientas Lex y Yacc

Lex y Yacc son herramientas ampliamente utilizadas en la construcción de compiladores. Lex se encarga de generar analizadores léxicos a partir de especificaciones basadas en expresiones regulares, mientras que Yacc permite generar analizadores sintácticos a partir de una gramática definida en notación BNF [2].

La integración de ambas herramientas permite desarrollar sistemas completos de análisis, donde Lex produce los tokens que son posteriormente procesados por Yacc, logrando validar la estructura de un lenguaje de manera automática [1].

3. Herramientas empleadas

Para el desarrollo de la presente práctica se utilizaron herramientas orientadas a la construcción de analizadores léxicos y sintácticos, las cuales permiten automatizar el reconocimiento y validación de estructuras en lenguajes de programación.

- **Flex:** Herramienta generadora de analizadores léxicos basada en expresiones regulares. Se utilizó para definir los patrones correspondientes a los tokens del lenguaje, tales como palabras reservadas, identificadores y operadores.
- **Yacc:** Generador de analizadores sintácticos que permite definir gramáticas mediante reglas de producción. Se empleó para validar la estructura del condicional `if - then` a partir de los tokens generados por el analizador léxico.
- **Flex Windows:** Entorno de ejecución utilizado para compilar y ejecutar los programas desarrollados con Flex y Yacc en un sistema operativo Windows, facilitando la integración de ambas herramientas.

4. Objetivos

Objetivo general

Desarrollar un analizador léxico y sintáctico utilizando las herramientas Flex y Yacc, capaz de reconocer y validar la estructura condicional `if - then`, permitiendo la identificación de errores sintácticos en un lenguaje de programación.

Objetivos específicos

- Aplicar los conceptos de autómatas y gramáticas libres de contexto en la construcción de un analizador de lenguaje.
- Diseñar un analizador léxico en Flex que permita identificar correctamente los tokens asociados a palabras reservadas, identificadores y operadores.
- Implementar un analizador sintáctico en Yacc que valide la correcta estructura de la sentencia `if - then` mediante reglas de producción.
- Integrar el analizador léxico y sintáctico para el procesamiento completo de entradas de código.
- Realizar pruebas con ejemplos correctos e incorrectos para verificar el funcionamiento del sistema y la detección de errores.
- Analizar los resultados obtenidos para evaluar la efectividad del analizador desarrollado.

5. Desarrollo

Desarrollo de la práctica

Diseño del analizador léxico

Para el análisis léxico se utilizó la herramienta Flex, mediante la cual se definieron expresiones regulares que permiten reconocer los distintos componentes del lenguaje. Se establecieron patrones para identificar palabras reservadas como **if**, **then**, **else**, ciclos (**for**, **while**) y estructuras como **switch**.

Asimismo, se incluyeron reglas para el reconocimiento de identificadores, números en diferentes formatos (enteros, flotantes, binarios, octales y hexadecimales), cadenas de texto y caracteres. También se definieron operadores relacionales y operadores simples, así como símbolos especiales como paréntesis, llaves y punto y coma.

El analizador léxico ignora espacios en blanco y comentarios (tanto de una línea como de bloque), permitiendo procesar únicamente los elementos relevantes del código fuente. En caso de encontrar un símbolo no reconocido, se genera un mensaje de error léxico indicando el elemento inválido.

Definición de tokens

Los tokens generados por el analizador léxico son utilizados por el analizador sintáctico. Entre los principales tokens definidos se encuentran:

- Palabras reservadas: IF, THEN, ELSE, FOR, WHILE, SWITCH, CASE, DEFAULT, BREAK.
- Tipos de datos: INT, FLOAT, CHAR.
- Identificadores y constantes: ID, NUM, STRING, CHARLIT.
- Operadores relacionales: RELOP.
- Símbolos especiales: paréntesis, llaves, operadores y signos de puntuación.

Estos tokens permiten abstraer la complejidad del código fuente y facilitan la validación sintáctica.

Diseño del analizador sintáctico

El análisis sintáctico se implementó utilizando Yacc, donde se definió una gramática libre de contexto capaz de reconocer distintas estructuras de control y sentencias básicas de un lenguaje de programación.

La gramática principal parte del símbolo inicial **programa**, el cual está compuesto por una lista de sentencias. A su vez, las sentencias pueden ser declaraciones, asignaciones, estructuras condicionales, ciclos, estructuras **switch** o instrucciones de impresión.

Se definieron reglas específicas para validar la estructura del condicional **if - then**, permitiendo tanto la forma simple como la variante con **else**. Además, se incluyeron reglas para ciclos **while** y **for**, así como para estructuras **switch-case**.

Validación de expresiones y bloques

Las condiciones dentro de las estructuras de control se definieron mediante expresiones que combinan identificadores y operadores relacionales. Asimismo, se implementó el concepto de bloque, el cual puede estar delimitado por llaves o consistir en una sola sentencia.

Esto permite que el analizador reconozca estructuras anidadas y valide correctamente la organización del código.

Manejo de errores

Se implementó una función de manejo de errores sintácticos mediante `yyerror`, la cual escribe mensajes en un archivo de salida. De esta manera, cuando el analizador detecta una estructura inválida, se genera un mensaje indicando la presencia de un error sintáctico.

Por otro lado, el analizador léxico también reporta errores cuando encuentra símbolos no reconocidos, proporcionando retroalimentación adicional sobre el código analizado.

Procesamiento de archivos de prueba

El programa principal fue diseñado para analizar dos archivos de entrada: uno con código correcto y otro con errores sintácticos. Antes de iniciar el análisis, se limpia el archivo de resultados para evitar mezclar ejecuciones previas.

Posteriormente, se procesa cada archivo de manera independiente, mostrando en consola y escribiendo en el archivo de salida si el análisis fue exitoso o si se detectaron errores.

Código del analizador léxico (Flex)

A continuación se muestra el código desarrollado en Flex para el reconocimiento de tokens:

```
%option noyywrap

%{
    #include <stdio.h>
    #include "y.tab.h"
}%

DIGITO      [0-9]
LETRA       [a-zA-Z]
ID          [a-zA-Z_] [a-zA-Z0-9_]*

INTEGERLIT  0|[1-9][0-9_]*
FLOATLIT    [0-9][0-9_]*\.[0-9][0-9_]*([eE][+-]?[0-9]+)?

HEXADECIMALLIT  0[xX][0-9a-fA-F][0-9a-fA-F_]*
BINARYLIT      0[bB][01][01_]*
OCTALLIT       0[0-7]+
```



```

STRINGLIT  \"([^\\"\\|\\.)*\"
CHARLIT    \'([^\'\\|\\.)\',

SIMPLEOP    [+\\-*/%<>!&\\|^~?]

LINECOMMENT  "//".*
BLOCKCOMMENT  "/*"([^\*]|\\*+[^*/])*\*+\\//

RELOP      "=="|"!="| "<="| ">="| "<"| ">"

```

```
%%
```

```

"if"      { return IF; }
"then"    { return THEN; }
"else"    { return ELSE; }

"for"     { return FOR; }
"while"   { return WHILE; }

"switch"  { return SWITCH; }
"case"    { return CASE; }
"default" { return DEFAULT; }
"break"   { return BREAK; }

"int"     { return INT; }
"float"   { return FLOAT; }
"char"    { return CHAR; }

"printf"  { return PRINTF; }

"System"  { return SYSTEM; }
"out"     { return OUT; }
"println" { return PRINTLN; }

```

```

{INTEGERLIT} { return NUM; }
{FLOATLIT}   { return NUM; }
{HEXADECIMALLIT} { return NUM; }
{BINARYLIT}  { return NUM; }
{OCTALLIT}   { return NUM; }

{STRINGLIT}  { return STRING; }
{CHARLIT}    { return CHARLIT; }

```

```

{RELOP}      { return RELOP; }
{SIMPLEOP}   { return yytext[0]; }

```

```

"("      { return '('; }
")"      { return ')'; }
"{"      { return '{'; }

```

```

"}"      { return '}' ; }
";"      { return ';' ; }
","      { return ',' ; }
":"      { return ':' ; }
"."      { return '.' ; }
"="      { return '=' ; }

{ID}      { return ID; }

{LINECOMMENT} { }
{BLOCKCOMMENT} { }

[ \t\n]+  { }

. {
    printf("Error léxico: %s\n", yytext);
}

%%

```

Código del analizador sintáctico (Yacc)

El siguiente código corresponde a la implementación del analizador sintáctico utilizando Yacc:

```

%{
#include <stdio.h>
#include <stdlib.h>

extern FILE *yyin;
extern FILE *yyout;

int yylex();
void yyerror(const char *s);
}%

%token IF THEN ELSE FOR WHILE SWITCH CASE DEFAULT BREAK
%token INT FLOAT CHAR
%token PRINTF SYSTEM OUT PRINTLN
%token ID NUM STRING CHARLIT
%token RELOP
%nonassoc THEN
%nonassoc ELSE

%%

programa:
    lista_sentencias
    ;

lista_sentencias:
    lista_sentencias sentencia
    | sentencia

```

```

;

sentencia:
    declaracion
    | asignacion
    | if_stmt
    | while_stmt
    | for_stmt
    | switch_stmt
    | print_stmt
;

declaracion:
    tipo ID ';'
;

tipo:
    INT
    | FLOAT
    | CHAR
;

asignacion:
    ID '=' NUM ';'
;

if_stmt:
    IF '(' condicion ')' THEN bloque
    | IF '(' condicion ')' THEN bloque ELSE bloque
;

while_stmt:
    WHILE '(' condicion ')' bloque
;

for_stmt:
    FOR '(' asignacion condicion ';' asignacion ')' bloque
;

switch_stmt:
    SWITCH '(' ID ')' '{' lista_casos '}'
;

lista_casos:
    lista_casos caso
    | caso
;

caso:
    CASE NUM ':' lista_sentencias BREAK ';'
    | DEFAULT ':' lista_sentencias
;

print_stmt:

```

```

        PRINTF '(' STRING ')' ' ';
    | SYSTEM '.' OUT '.' PRINTLN '(' STRING ')' ' ';
    ;

condicion:
    ID RELOP ID
    ;

bloque:
    '{' lista_sentencias '}'
    | sentencia
    ;

%%

void yyerror(const char *s) {
    if (yyout) {
        fprintf(yyout, "Error sintáctico: %s\n", s);
    }
}

int main(int argc, char *argv[]) {

    FILE *archivo1, *archivo2;

    //Borrar resultado previo
    yyout = fopen("D:\\Flex Windows\\EditPlusPortable\\Practica7\\resultado.txt", "w");
    if (!yyout) {
        perror("Error al abrir archivo de salida");
        return 1;
    }

    //Correcto
    archivo1 = fopen("D:\\Flex Windows\\EditPlusPortable\\Practica7\\sintactico_correcto.txt", "r");
    if (!archivo1) {
        perror("Error al abrir archivo correcto");
        return 1;
    }

    yyin = archivo1;

    fprintf(yyout, "Analizando archivo CORRECTO:\n");

    if (yyparse() == 0) {
        printf("Archivo correcto: OK\n");
        fprintf(yyout, "Análisis sintáctico correcto.\n\n");
    } else {
        printf("Archivo correcto: ERROR\n");
        fprintf(yyout, "Análisis sintáctico con errores.\n\n");
    }

    fclose(archivo1);
}

```

```

yyrestart(NULL);

//Archivo incorrecto
archivo2 = fopen("D:\\Flex Windows\\EditPlusPortable\\Practica7\\sintactico_incorrecto.txt", "r");
if (!archivo2) {
    perror("Error al abrir archivo incorrecto");
    return 1;
}

yyin = archivo2;

fprintf(yyout, "Analizando archivo INCORRECTO:\n");

if (yyparse() == 0) {
    printf("Archivo incorrecto: OK\n");
    fprintf(yyout, "Análisis sintáctico correcto.\n");
} else {
    printf("Archivo incorrecto: ERROR\n");
    fprintf(yyout, "Análisis sintáctico con errores.\n");
}

fclose(archivo2);
fclose(yyout);

return 0;
}

```

Archivos de prueba

Para validar el funcionamiento del analizador, se utilizaron los siguientes archivos:

12.9.1 Ejemplo correcto

```

int x;
int y;

if (x > y) then {
    x = 5;
} else {
    x = 10;
}

while (x < y) {
    x = 1;
}

printf("Hola");

System.out.println("Mundo");

```

12.9.2 Ejemplo incorrecto

```
int x
int y;

if (x > y) then {
    x = 5
} else
    x = 10;
}

while (x < y)
{
    x = 1;
}

printf("Hola")

System.out.println("Mundo");
```

6. Resultados

Resultados

Para evaluar el funcionamiento del analizador léxico y sintáctico desarrollado, se realizaron pruebas utilizando dos archivos de entrada: uno con estructuras correctamente definidas y otro con errores sintácticos intencionales.

Análisis de archivo correcto

El archivo de entrada correcto contiene declaraciones de variables, estructuras condicionales `if - then - else`, ciclos `while` y sentencias de impresión. Al procesar este archivo, el analizador léxico identificó correctamente todos los tokens definidos, mientras que el analizador sintáctico validó la estructura general del programa sin detectar errores.

Como resultado, el sistema generó un mensaje indicando que el análisis sintáctico fue exitoso, lo cual confirma que la gramática definida en Yacc reconoce adecuadamente las construcciones válidas del lenguaje implementado.

Análisis de archivo incorrecto

El archivo incorrecto fue diseñado con diversos errores sintácticos, tales como la omisión de puntos y coma, estructuras incompletas y errores en la delimitación de bloques.

Durante el análisis, el analizador léxico logró identificar los tokens válidos presentes en el archivo; sin embargo, el analizador sintáctico detectó inconsistencias en la estructura del programa, generando mensajes de error mediante la función `yyerror`.

El resultado obtenido indica que el análisis sintáctico contiene errores, lo cual demuestra la capacidad del sistema para identificar estructuras inválidas y reportarlas adecuadamente.

Salida generada

Los resultados del análisis fueron almacenados en un archivo de texto, en el cual se muestra el estado del análisis para cada archivo procesado. Además, se imprimieron mensajes en consola que permiten al usuario conocer de manera inmediata si el archivo es correcto o contiene errores.

En la Figura 1 se observan los archivos ingresados así como los resultados obtenidos correspondientemente

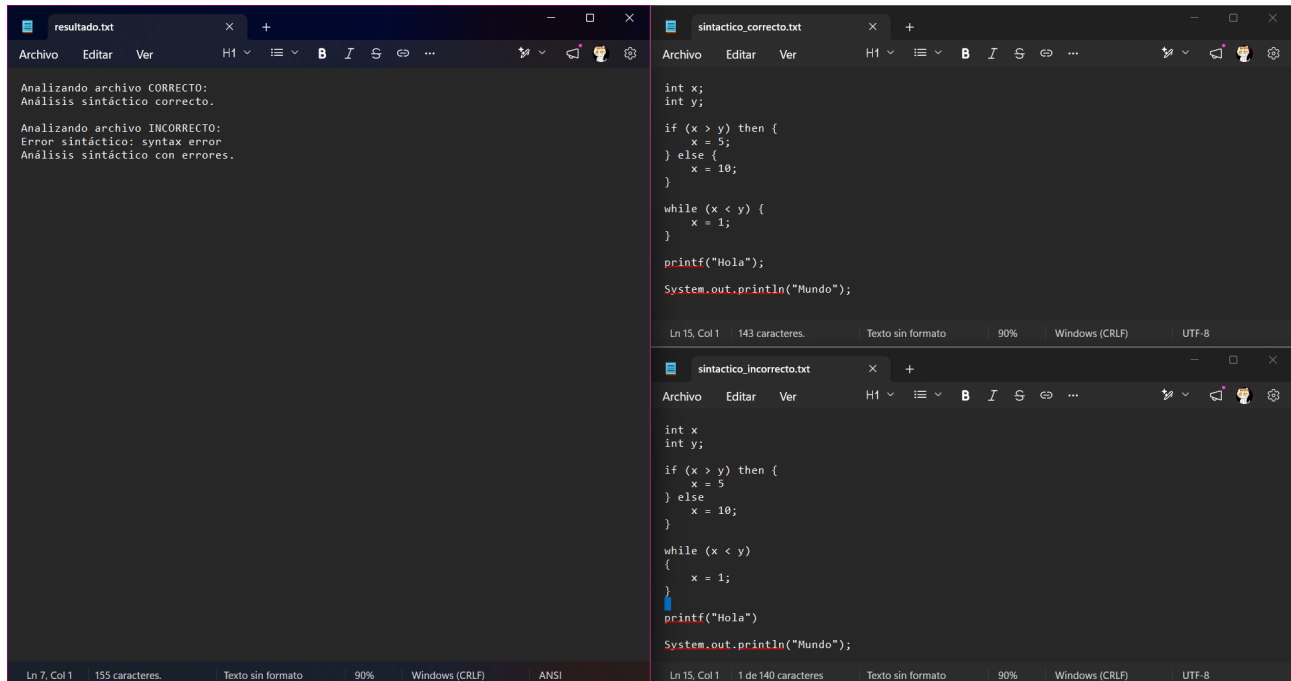


Figure 1: Resultados

Interpretación de resultados

A partir de las pruebas realizadas, se observa que el analizador desarrollado cumple con su objetivo, ya que es capaz de diferenciar entre estructuras sintácticamente correctas e incorrectas. La integración entre el analizador léxico y sintáctico permite un procesamiento adecuado de las entradas, validando tanto la correcta formación de tokens como su organización dentro de la gramática definida.

Asimismo, los errores detectados corresponden a inconsistencias reales en la estructura del código, lo que confirma que la gramática implementada en Yacc es funcional y adecuada para el reconocimiento de la estructura `if - then` y otras construcciones incluidas.

7. Cuestionario

Pregunta 1: ¿Cuáles son las principales ventajas de usar gramáticas de contexto libre para definir estructuras como `if...then` en lenguajes de programación?

Respuesta: Las gramáticas de contexto libre permiten definir de manera clara y estructurada la sintaxis de un lenguaje de programación, lo que facilita la representación de construcciones como `if...then`. Una de sus principales ventajas es que pueden describir estructuras anidadas y jerárquicas, algo fundamental en este tipo de sentencias.

Además, permiten separar la lógica del reconocimiento sintáctico de la implementación, lo que hace más sencillo modificar o extender la gramática sin afectar directamente el funcionamiento del analizador. También son compatibles con herramientas como Yacc, lo que facilita la generación automática de analizadores sintácticos eficientes.

Pregunta 2: ¿Qué tipo de errores fueron detectados en los ejemplos incorrectos y cómo influyeron las

gramáticas definidas para lograr su identificación?

Respuesta: Los errores detectados en las pruebas fueron principalmente de tipo sintáctico, como la omisión de puntos y coma, estructuras incompletas y delimitación incorrecta de bloques. Por ejemplo, en el archivo incorrecto se presentan instrucciones sin punto y coma y bloques mal cerrados, lo cual rompe la estructura esperada por la gramática.

La gramática influye directamente al definir las reglas que deben cumplirse para que una estructura sea válida. Cuando el analizador sintáctico encuentra una secuencia de tokens que no coincide con ninguna producción, se genera un error. Esto ocurre porque el parser no puede continuar con una derivación válida, lo que permite detectar con precisión en qué punto la entrada deja de ser correcta.

Pregunta 3: ¿Cómo contribuye el uso de derivaciones por la izquierda o por la derecha en la construcción de un árbol sintáctico para validar estructuras condicionales?

Respuesta: Las derivaciones por la izquierda y por la derecha permiten definir el orden en el que se aplican las reglas de producción durante el análisis sintáctico. En particular, estas derivaciones ayudan a construir el árbol sintáctico que representa la estructura del programa.

En herramientas como Yacc, se utilizan derivaciones por la derecha (análisis ascendente), lo que significa que el árbol se construye desde las hojas hacia la raíz. Esto permite reconocer primero los elementos más simples y luego agruparlos en estructuras más complejas, como las sentencias `if...then`. Gracias a este proceso, es posible validar correctamente la jerarquía y organización de las instrucciones.

Pregunta 4: ¿Qué ajustes serían necesarios en la gramática para permitir el reconocimiento de estructuras condicionales más complejas, como `if...then...else`?

Respuesta: Para permitir el reconocimiento de estructuras más complejas, sería necesario ampliar las reglas de producción en la gramática. En el caso de `if...then...else`, se deben definir producciones que contemplen tanto la versión simple como la que incluye la cláusula `else`.

Además, es importante manejar correctamente la ambigüedad conocida como “dangling else”, lo cual se puede resolver mediante el uso de precedencias o reglas específicas en Yacc, como el uso de `nonassoc THEN` y `nonassoc ELSE`. También podrían agregarse nuevas reglas para soportar múltiples condiciones, operadores lógicos o estructuras anidadas, haciendo el lenguaje más completo y robusto.

8. Conclusiones

El desarrollo de la presente práctica permitió comprender de manera más clara el funcionamiento interno de las etapas de análisis léxico y sintáctico dentro de un compilador. A través de la implementación con Flex y Yacc, se logró observar cómo un lenguaje de programación puede ser interpretado a partir de reglas formales, partiendo desde la identificación de tokens hasta la validación de estructuras completas.

Uno de los aspectos más relevantes fue la aplicación de gramáticas libres de contexto para definir la estructura de sentencias como `if - then`, lo cual evidenció la importancia de establecer reglas bien definidas para evitar ambigüedades y garantizar un análisis correcto. Asimismo, el uso de herramientas como Yacc facilitó la construcción del analizador sintáctico, permitiendo detectar errores de manera eficiente cuando la entrada no cumple con la gramática especificada.

Las pruebas realizadas con archivos correctos e incorrectos demostraron que el sistema desarrollado es capaz de distinguir entre estructuras válidas e inválidas, identificando errores como la omisión de delimitadores o la mala organización de bloques. Esto refuerza la utilidad de los analizadores en la validación de código fuente dentro de los lenguajes de programación.

Finalmente, esta práctica permitió reforzar conceptos teóricos como autómatas, expresiones regulares y derivaciones, llevándolos a una aplicación práctica. Además, abre la posibilidad de extender el analizador para reconocer estructuras más complejas, lo que resulta fundamental en el desarrollo de compiladores más completos.

Referencias Bibliográficas

References

- [1] Aho, A. V., Lam, M. S., Sethi, R., Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. Pearson.
- [2] Levine, J. R., Mason, T., Brown, D. (1992). *Lex & Yacc*. O'Reilly Media.
- [3] Hopcroft, J. E., Motwani, R., Ullman, J. D. (2007). *Introduction to Automata Theory, Languages, and Computation*. Pearson.
- [4] Sipser, M. (2013). *Introduction to the Theory of Computation*. Cengage Learning.